

PRA – TP1 – Jeu de Morpion

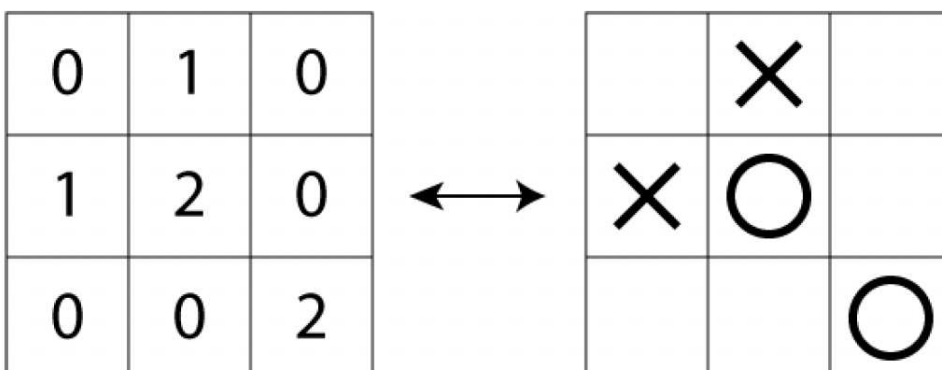
Dans ce TP, vous devez implémenter le jeu du Morpion. Ce jeu se dispute à deux sur un plateau composé de 9 cases. A tour de rôle, chaque joueur inscrit un symbole (croix ou rond) dans une case vide. Le premier joueur parvenant à aligner trois symboles identiques remporte la partie.

Description de la classe `ModeleMorpions`

Cette classe implémente le noyau fonctionnel du jeu, c'est-à-dire l'état d'une partie de morpions suite à 0, 1 ou plusieurs coups joués. Pour simplifier la programmation, on supposera que le plateau du jeu est modélisé par une matrice 3x3 composée des valeurs de type énuméré suivant :

```
1 public enum Owner {  
2     NONE, FIRST, SECOND;  
3     public Owner opposite() {  
4         switch (this) {  
5             case SECOND: return FIRST;  
6             case FIRST:  return SECOND;  
7             default:     return NONE;  
8         }  
9     }  
10 }
```

Au départ, les éléments de cette matrice sont initialisés à `NONE`. Lorsqu'un joueur inscrit son symbole dans une case du plateau, l'élément correspondant de la matrice se voit attribuer le numéro du joueur courant.



Ce qui est à faire

Au cours de ce TP vous serez tour à tour développeur et testeur, en vous appuyant sur une source unique, à savoir l'interface `AbstractTicTacToe` constituée bien sûr de code Java mais aussi de spécifications prenant la forme de commentaires décrivant pour chaque méthode les paramètres, les préconditions et les résultats attendus. Cette interface sera envoyé par WhatsApp.

Dans ce qui suit le travail d'implémentation de l'interface (le codage à proprement parler) est expliqué avant le test pour des raisons de clarté. Néanmoins il est conseillé de réaliser la classe de test avant l'implémentation, car cela vous permettra de mieux comprendre les spécifications avant de les coder.

1. Réaliser sous JUnit 5 la classe de test `TicTacToeTest`.

Cette classe est chargée de vérifier dynamiquement qu'une implémentation donnée de l'interface `AbstractTicTacToe` produit des résultats conformes aux spécifications exprimées dans les commentaires de cette même interface.

(a) Mise en place de l'environnement Java/JUnit 5

- dans Java Build Path, vérifiez dans Classpath > Libraries que JUnit 5 est bien présent ;
- incluez le Jar contenant le package par un Add External Jars sur le fichier `pra-tp3.jar` que vous aurez téléchargé entre temps depuis Teams ;
- validez la création du projet par Finish ;
- créez un paquetage, puis une classe `TicTacToeTest` dans ce paquetage. Copiez-y le code de `TicTacToeTest` ;
- lancez `TicTacToeTest` par Run as JUnit Test ;
- modifiez votre classe `TicTacToeTest` de manière à ce qu'une seule classe `TicTacToeVX` (ainsi que votre propre implémentation, quand vous l'aurez validée) réussisse cette classe de test à 100%. Toutes les autres classes `TicTacToeVY` ne doivent pas la réussir.

(b) Définir et mettre en œuvre une stratégie de test.

Même s'il n'existe pas de " méthode " infaillible pour écrire une classe de test, on procède généralement comme suit :

- i. Cibler les objectifs du test : cela correspond souvent aux méthodes, mais pas nécessairement (par la suite chaque objectif donnera lieu à une méthode de test, annotée par `@Test`).

Dans le cas de ce TP cela donnerait par exemple ceci :

```
// Une méthode de test par méthode de la classe testée :
@Test
public void testGetJoueur()
    // scénarios vérifiant le bon fonctionnement de getTurn()
@Test
public void testGetVainqueur()
    // scénarios vérifiant le bon fonctionnement de getWinner()
    // et ainsi de suite pour numberOfRounds(), \dots ,
    // jusqu'à play()
```

\dots ou encore cela :

```
@Test
```

```

public void testInit()
    // Scénario(s) vérifiant l'état du jeu avant le premier coup :
    // * non-fin de partie
    // * cases accessibles
@Test
public void testPremierCoup()
    // Scénario vérifiant le premier coup joué, notamment :
    // * position correcte ou non
    // * non-fin de partie
    // * identité du premier joueur
@Test
public void testPartiePasFinie()
    // Scénario explorant les situations de non-fin de partie,
    // avec vérification systématique de gameOver() == false
@Test
public void testControle()
    // Scénario tentant divers coups non autorisés,
    // avec vérification systématique de legalMove()
@Test
public void testFinPartie()
    // Scénarios explorant les situations de fin de partie,
    // avec vérification systématique de gameOver() et de
    // getWinner() :
    // * alignement horizontal (grille pleine ou non)
    // * alignement vertical (grille pleine ou non)
    // * alignement diagonal (grille pleine ou non)
    // et ainsi de suite...

```

- ii. Écrire pour chaque objectif (méthode de test) un ou plusieurs scénarios tels que celui-ci, que l'on pourrait envisager pour la méthode testPartiePasFinie() :

initialiser la partie

joueur 1 : (0,2)

joueur 2 : (0,1)

joueur 1 : (1,0)

joueur 2 : (0,0)

joueur 1 : (1,1)

joueur 2 : (2,0)

joueur 1 : (2,1)

joueur 2 : (1,2)

joueur 1 : (2,2)

- iii. Pour chaque changement d'état d'un objet mis en test, écrire un « oracle » prédisant dans quel état doit se trouver cet objet. Dans notre exemple, après chaque étape du scénario excepté le tout dernier, on affirme que la partie ne peut être finie, car il reste des cases vides, et il n'existe aucune rangée gagnante. Après le dernier coup joué en revanche, l'oracle consiste à affirmer que la partie est un match nul.

- iv. Coder en Java chaque scénario formé d’actions et d’oracles : les actions sont codées de manière « classique », à l’instar d’un simple main, tandis que chaque oracle est traduit par l’appel d’une des méthodes de classe de org.junit.Assert : assertEquals(), assertTrue(), etc. Il faut veiller à ce que chacun de ces appels commence par un message en langage naturel décrivant de manière précise la situation attendue, tandis que les 2ème et 3ème paramètres expriment respectivement le résultat attendu (true par défaut dans le cas de assertTrue) et le résultat observé depuis l’objet testé.

Voici des extraits de code Java réalisant le début et la fin de ce scénario :

```
1  partieEnTest = new TicTacToeModel();
2  partieEnTest.play(0, 2);
3  assertTrue(!partieEnTest.gameOver()),
4      "La partie ne peut être jouée en un seul coup") ;
5  partieEnTest.play(1, 2) ;
6
7  // etc.
8
9  assertTrue (partieEnTest.gameOver(),
10      "Après 9 coups la partie est forcément terminée");
```

2. Réaliser une classe TicTacToeModel respectant les spécifications

Pour éviter les classes avec le même nom, il faut créer un nouveau package.

Comme pour la classe Grid du TP précédent, il vous faudra déclarer une variable d’instance à deux dimensions, de type Owner[][] pour être précis.